

GNU Backgammon Android Port

clavierhaus.at

June 2026

Contents

1	GNU Backgammon Android Port — MASTER v0.8.6	2
1.1	1. Introduction & Project Objective	2
1.1.1	1.1 Scope	2
1.1.2	1.2 App Identity	2
1.2	2. Narrative of Progress	3
1.2.1	Phase 1: Monolithic Analysis & Prototype	3
1.2.2	Phase 2: Redesign — Deterministic Engineering	3
1.2.3	Phase 3: The Pristine Headless Build (Completed)	3
1.2.4	Phase 4: GLib Cross-Compilation for Android (Completed)	3
1.2.5	Phase 5: JNI Bridge & libgnubg-engine.so (Completed)	3
1.2.6	Phase 6: NEON Acceleration & Device Verification (Completed)	3
1.2.7	Phase 7: Cube Decisions & Rollout (Completed)	3
1.2.8	Phase 8: Full Game Logic Layer (Completed)	3
1.2.9	Phase 9: Multi-threaded Rollout & SGF JNI API (Completed)	3
1.3	3. Architecture	4
1.3.1	3.1 Core Principle	4
1.3.2	3.2 Key gnubg Functions Used	4
1.3.3	3.3 JNI Bridge	4
1.3.4	3.4 NextTurn() and TurnDone()	4
1.3.5	3.5 Board Encoding Reference	4
1.4	4. Current JNI API	5
1.5	5. Version v0.8.6 — Milestone	5
1.5.1	5.1 Assessment	5
1.5.2	5.2 What Works	6
1.5.3	5.3 What Does Not Yet Work	6
1.5.4	5.4 Key Lessons Learned	6
1.5.5	5.5 Immediate Next Steps (v0.8.6)	7
1.6	6. Version v0.8.6 — Engine Dice, Roll Button, Gameplay Stability	7
1.6.1	6.1 Milestone	7
1.6.2	6.2 Delivered	7
1.6.3	6.3 Architecture Notes	8
1.6.4	6.4 Immediate Next Steps (v0.8.7)	8

1. GNU Backgammon Android Port — MASTER v0.8.6

GNU Backgammon by clavierhaus.at

clavierhaus.at · Vienna · Austria

June 2026

1.1 1. Introduction & Project Objective

The objective of this project is to produce a **true port** of GNU Backgammon to the Android platform — not a stripped-down evaluator, but a complete port that mirrors 100% of the functionality of the PC version. This includes full game management, SGF import/export, analysis, rollout, cube decisions, tutor mode, and the complete match state machine.

The primary constraint is the monolithic nature of the original gnubg desktop codebase, which is tightly coupled with GTK/GNOME UI components, file I/O, global configuration state, and platform-specific threading primitives. The approach is to compile the genuine gnubg engine source — every file that is portable — and replace only what is inherently GTK-specific with Android equivalents via a clean architectural boundary: `android-app.c`.

The key architectural insight: gnubg's codebase separates cleanly into three layers:

1. **Mathematical engine** (`eval.c`, `rollout.c`, `dice.c`, `bearoff.c` etc.) — pure computation, zero UI dependencies, 100% portable
2. **Game logic layer** (`play.c`, `analysis.c`, `sgf.c`, `set.c`, `format.c` etc.) — all GTK references are behind `#if defined(USE_GTK)` guards, portable when compiled without `USE_GTK`
3. **GTK application shell** (`gnubg.c`, `progress.c`, `gtk/*.c`) — not portable, replaced by `android-app.c`

Every stub that silently drops functionality is a hole in the port. The goal is to minimise stubs to only what is genuinely GTK-specific rendering code.

Architecture principle: gnubg is the authority. No game logic is reimplemented in Kotlin or the JNI bridge. The Android layer calls gnubg's own functions and reads `ms` (the global matchstate) directly. The UI reflects what the engine owns.

1.1.1 1.1 Scope

This document covers the complete build history, all architectural decisions, every obstacle encountered and its resolution, and the exact current state of the project as of v0.8.6. It is intended as a working reference for any software engineer picking up or contributing to this project.

1.1.2 1.2 App Identity

Property	Value
App name	GNU Backgammon by clavierhaus.at
Android package	<code>com.clavierhaus.gnubg</code>
Minimum Android API	31 (Android 12.0, Snow Cone)
Target ABI	<code>arm64-v8a</code>
Engine source	GNU Backgammon upstream (gnubg.org)
Repository	https://github.com/clavierhaus/gnubg-android

1.2 2. Narrative of Progress

1.2.1 Phase 1: Monolithic Analysis & Prototype

Initial efforts focused on analysing the upstream GNU Backgammon source. Attempts to build the full desktop environment proved unsustainable due to deep dependencies on GTK, GLib, and global UI state variables (ms, ap, positions). A “Frankenstein” build approach established that the mathematical evaluation logic was sound in isolation, but produced a fragile, non-portable build environment.

1.2.2 Phase 2: Redesign — Deterministic Engineering

Phase 2 pivoted to a “Clean-Room” architecture. The upstream source acts as a read-only reference. The engine-core/ directory contains only the verified, essential components. Orchestration scripts (pull_dependencies.sh, patch_engine.sh, build_all.sh) and the Patch Registry were established.

1.2.3 Phase 3: The Pristine Headless Build (Completed)

Phase 3 achieved a fully hermetic host build of the mathematical core, confirming that the engine compiles and links cleanly on Linux when detached from the GTK/GNOME infrastructure.

1.2.4 Phase 4: GLib Cross-Compilation for Android (Completed)

GLib 2.88.1 was cross-compiled for arm64-v8a Android API 28 from the Fedora 44 SRPM, providing the full GLib API that gnubg requires with no behavioural deviations from upstream.

1.2.5 Phase 5: JNI Bridge & libgnubg-engine.so (Completed)

The JNI bridge layer (native-lib.c, stubs.c, Engine.kt) and CMake build pipeline were constructed, producing libgnubg-engine.so verified as ELF 64-bit ARM aarch64.

1.2.6 Phase 6: NEON Acceleration & Device Verification (Completed)

ARM NEON SIMD acceleration was enabled and the engine verified on a Pixel 8 Pro (aarch64, Android 16). Opening position evaluation and FindBestMove 3-1 returning 8/5 6/5 confirmed correct.

1.2.7 Phase 7: Cube Decisions & Rollout (Completed)

Engine.cubeDecision() and Engine.rollout() JNI entry points added and verified. Multi-threaded rollout via GThreadPool added in Phase 9.

1.2.8 Phase 8: Full Game Logic Layer (Completed)

play.c, analysis.c, sgf.c and all dependencies compiled into libgnubg-engine.so. The android-app.c architecture provides all GTK replacements.

1.2.9 Phase 9: Multi-threaded Rollout & SGF JNI API (Completed)

Production-quality parallel rollout and SGF load/save JNI entry points complete the engine feature set.

1.3 3. Architecture

1.3.1 3.1 Core Principle

gnubg's `ms` (global matchstate) is the single source of truth. All game state — board, dice, turn, game status — is owned by gnubg. The Android layer does three things only:

1. Call gnubg functions (`CommandRoll`, `CommandMove`, `ApplySubMove`, etc.)
2. Read `ms` fields (`ms.anBoard`, `ms.anDice`, `ms.fTurn`, `ms.gs`) after each call
3. Render what the engine reports

No move computation, no dice tracking, no legal move determination is performed in Kotlin. The engine exclusively determines what is legal.

1.3.2 3.2 Key gnubg Functions Used

gnubg Function	Purpose
<code>CommandNewMatch("1")</code>	Start a new 1-point match
<code>CommandNewGame(NULL)</code>	Opening roll, sets <code>ms.fTurn</code> to winner
<code>CommandRoll(NULL)</code>	Roll dice; for <code>PLAYER_GNU</code> also runs <code>ComputerTurn</code>
<code>CommandMove(str)</code>	Apply move string, record <code>MOVE_NORMAL</code> , call <code>TurnDone()</code>
<code>NextTurn(TRUE)</code>	Advance <code>ms.fTurn</code> ; called inside <code>applyMoveString</code>
<code>ApplySubMove(board, iSrc, nRoll, TRUE)</code>	Apply one sub-move for display; no match record
<code>GenerateMoves(&ml, board, d0, d1, fPartial)</code>	Generate all legal moves
<code>FormatMove(sz, board, anMove)</code>	Format move as gnubg point notation
<code>PipCount(board, anPip)</code>	Compute pip counts
<code>SwapSides(board)</code>	Flip <code>anBoard[0]</code> ↔ <code>anBoard[1]</code>

1.3.3 3.3 JNI Bridge

The JNI bridge (`native-lib.c`) is a thin wrapper layer. Each JNI function calls one or more gnubg functions and reads `ms` directly. No logic originates in the bridge. The bridge also implements `gnubg_on_board_changed()` (called by gnubg on every board update) with a simple LOGI call for diagnostics.

1.3.4 3.4 NextTurn() and TurnDone()

This is the most significant architectural subtlety. In the GTK version, `TurnDone()` sets `fNextTurn = TRUE` and the GTK idle loop calls `NextTurnNotify()` → `NextTurn()` to advance `ms.fTurn`. On Android there is no GTK idle loop.

Solution: `applyMoveString` calls `NextTurn(TRUE)` explicitly after `CommandMove`. `NextTurn` advances `ms.fTurn` and triggers `ComputerTurn` if the next player is `PLAYER_GNU`.

1.3.5 3.5 Board Encoding Reference

gnubg's `TanBoard` `anBoard[2][25]` as used in this port:

Array	Player	UI point mapping
<code>anBoard[0][i]</code>	Engine/dark	UI point <code>i+1</code> (top, moves right to left)
<code>anBoard[1][i]</code>	Human/light	UI point <code>24-i</code> (bottom, moves left to right)
<code>anBoard[0][24]</code>	Engine checkers on bar	Bar (top half)

Array	Player	UI point mapping
<code>anBoard[1][24]</code>	Human checkers on bar	Bar (bottom half)

gnubg always places the moving player's checkers in `anBoard[1]`. When `ms.fTurn == 1` (engine moving), the board is swapped for display via `SwapSides` to maintain the fixed human-bottom perspective.

1.4 4. Current JNI API

Engine.* Kotlin call	JNI → gnubg mapping
<code>initialise(weightsPath)</code>	<code>EvalInitialise</code> , <code>gnubg_init_tld/rollout</code> , <code>ClearMatch</code>
<code>newGame()</code>	<code>CommandNewMatch("1") + CommandNewGame(NULL)</code>
<code>rollDice()</code>	<code>CommandRoll(NULL)</code> , returns <code>ms.anDice</code>
<code>getLegalMoves(board,d0,d1,fPartial)</code>	<code>GenerateMoves</code>
<code>applyMoveString(str)</code>	<code>CommandMove(str) + NextTurn(TRUE)</code>
<code>applySubMove(board,iSrc,nRoll)</code>	<code>ApplySubMove(...,TRUE)</code>
<code>findMove(oldBoard,curBoard,d0,d1)</code>	<code>GenerateMoves + PositionKey comparison + FormatMove</code>
<code>formatMove(board,anMove)</code>	<code>FormatMove</code>
<code>getMatchBoard()</code>	<code>ms.anBoard</code>
<code>getMatchDice()</code>	<code>ms.anDice</code>
<code>getMatchTurn()</code>	<code>ms.fTurn</code>
<code>getMatchStatus()</code>	<code>ms.gs</code>
<code>pipCount(board)</code>	<code>PipCount</code>
<code>swapBoard(board)</code>	<code>SwapSides</code>
<code>evaluatePosition(board)</code>	<code>EvaluatePosition</code>
<code>classifyPosition(board)</code>	<code>ClassifyPosition</code>
<code>cubeDecision(...)</code>	<code>GeneralCubeDecisionENoLocking + FindCubeDecision</code>
<code>rollout(board,trials)</code>	<code>gnubg_rollout</code> (multi-threaded)
<code>loadSGF(path)</code>	<code>CommandLoadMatch</code>
<code>saveSGF(path)</code>	<code>CommandSaveMatch</code>

1.5 5. Version v0.8.6 — Milestone

1.5.1 5.1 Assessment

Version v0.8.6 delivers a playable one-point game with correct gnubg move handling, sub-move interaction, and board rendering. The milestone is less advanced than originally projected (formerly labelled v0.9.5) due to a significant remediation effort required to remove non-gnubg game logic that had been introduced during earlier development. The remediation was necessary and correct: the resulting codebase is architecturally sound.

1.5.2 5.2 What Works

- **Full game loop:** Opening roll → human moves → commit → engine rolls and moves automatically → human rolls → repeat. Both players complete turns correctly across multiple games.
- **Opening roll:** `CommandNewGame` rolls one die per player, determines winner, sets `ms.fTurn` and `ms.anDice`. Human-wins branch enters `HUMAN_MOVING` immediately; engine-wins branch enters `WAITING_FOR_ROLL` after engine auto-plays.
- **Sub-move interaction:** Single tap on a source point applies one sub-move via `gnubg's ApplySubMove(board, iSrc, nRoll, TRUE)`, mirroring the single-click behaviour of `board_button_release` in `gtkboard.c`. First die tried, then second.
- **Commit:** Tap Commit button → `findMove(oldBoard, currentBoard, d0, d1)` locates the complete move by position key comparison (mirroring `update_move()` in `gtkboard.c`) → `CommandMove(str)` records and finalises the turn → `NextTurn(TRUE)` advances `ms.fTurn`.
- **Undo:** Tap Undo button → display board restored to `oldBoard` (board at start of human turn); `remainingDice` reset to original roll. Pure UI state; no engine call required since `ApplySubMove` never modifies `ms`.
- **Dice swap:** Tap dice area → swap die order in `remainingDice`. Mirrors clicking the dice in `gtkboard.c`.
- **Board rendering:** All 24 points, bar (human and engine), bearoff trays, pip counts, doubling cube, dice with used-die greying. Human checkers always at bottom regardless of whose turn it is.
- **Hit checker:** Checker hit onto bar displayed correctly in the bar area.
- **Doubles:** Four dice displayed; all four sub-moves playable.
- **Game over:** `ms.gs >= 2` detected; New Game button appears.
- **Commit/Undo button layout:** Drawn directly on the Canvas in `gnubg` board coordinate units, positioned below the dice in the tray gap of the right (human) half. Gap between Commit and Undo aligns vertically with the gap between the two dice. The entire ensemble (dice top row, buttons bottom row) is centred on the mid-point of the right half of the board.

1.5.3 5.3 What Does Not Yet Work

- **Doubling cube interaction** — cube offer, accept, pass not yet wired to UI
- **Bearoff checker display in tray** — checkers borne off are not yet rendered in the bearoff tray
- **Game over screen** — `gammon/backgammon` detection not yet surfaced in UI
- **SGF save/load UI** — JNI entry points exist but no UI
- **Player panel** — score, match length, player names not yet displayed
- **Analysis and tutor mode** — engine capability exists; UI not yet built
- **16KB page alignment** — `.so` not yet built with `-Wl, -z, max-page-size=16384` for Android 16 compatibility

1.5.4 5.4 Key Lessons Learned

gnubg source is the only authority. Any game logic not traceable to a specific `gnubg` source function is a liability. During this development cycle, several pieces of invented logic were introduced (cached dice, accumulated move strings, move history stacks, manual pip-count index swaps) that each caused subtle bugs. Every one was eventually traced to a divergence from `gnubg's` own logic. The remediation rule: before writing any code, read the corresponding `gnubg` source function and translate it directly.

The TurnDone/NextTurn split. `TurnDone()` does not advance `ms.fTurn`; it only sets `fNextTurn = TRUE`. `NextTurn()` does the advancement. In the GTK version this happens via the GLib idle loop. On Android, `NextTurn(TRUE)` must be called explicitly after `CommandMove`. Failure to do this leaves `ms.fTurn` pointing at the player who just moved.

ms.anDice is cleared by TurnDone(). `ms.anDice` is set by `CommandRoll` for `PLAYER_HUMAN` and remains valid until `TurnDone()` clears it. For `PLAYER_GNU`, `ComputerTurn` runs and `TurnDone()` fires before `CommandRoll` returns. Therefore `ms.anDice` is always 0 after an engine turn. Display dice must be tracked in `remainingDice` (Kotlin state), not read from `ms.anDice` after the fact.

ApplySubMove does not touch ms. `ApplySubMove(TanBoard, iSrc, nRoll, fCheckLegal)` operates purely on the board array passed to it. It never modifies `ms`. This makes it correct and safe for incremental display updates during human move selection. The final `CommandMove` then commits everything to `ms` atomically.

findMove via position key. After the human has applied sub-moves to the display board, the complete move must be identified for `CommandMove`. The correct method (taken directly from `update_move()` in `gtkboard.c`) is: `GenerateMoves` on the original board, then compare `PositionKey(currentBoard)` against `ml.amMoves[i].key` for each candidate. This avoids any manual move-string construction.

1.5.5 5.5 Immediate Next Steps (v0.8.6)

1. Bearoff checker display in right tray
2. Gammon/backgammon detection on game over (`ms.gs` values)
3. Player panel: score, match length, player names
4. Doubling cube UI: offer, accept, pass (wired to `CommandDouble`, `CommandTake`, `CommandDrop`)
5. Rebuild .so with `-Wl, -z, max-page-size=16384` for Android 16 16KB page alignment

1.6 6. Version v0.8.6 — Engine Dice, Roll Button, Gameplay Stability

1.6.1 6.1 Milestone

Version v0.8.6 completes the core visual game loop. Engine dice are now persistently displayed after each engine move. The Roll button appears on the board canvas in the right half, aligned with where the player's dice will appear. All dice and buttons are drawn in a single coordinate system (board units) with tap detection in the same system, guaranteeing pixel-perfect alignment on any device.

1.6.2 6.2 Delivered

Engine Dice Display Engine dice were previously invisible after each move because `ms.anDice` is cleared by `TurnDone()` immediately after the engine plays. The correct source is the move record: `getMoveRecordDice()` reads `pLastMove->p->anDice[0/1]`, which persists until the next move record is written. This mirrors exactly what gnubg's GTK board does for display.

Engine dice are rendered in the left half of the board, centred on the same horizontal line as the cube and the player's Roll button. Color: `Color.White.copy(alpha = 0.3f)` — same as spent player dice.

Roll Button on Canvas The Roll button is drawn directly on the Canvas in board-unit coordinates, centred in the right half of the board. Tap detection uses the same board-unit

coordinate system as all other board elements. This replaces the left-panel Roll button and makes the interaction spatially natural.

fAutoRoll = FALSE `fAutoRoll` was defined as `TRUE` in `android-app.c` line 494. With `fAutoRoll = TRUE`, `NextTurn()` automatically calls `CommandRoll(NULL)` for the human player immediately after the engine moves, bypassing the Roll button entirely. Setting `fAutoRoll = FALSE` restores correct behaviour.

Commit Re-entry Guard Multiple rapid taps on the Commit button queued multiple coroutines, each calling `applyMoveString` on the same stale state. Fixed by setting `phase = ENGINE_THINKING` immediately after `applyMoveString` succeeds, blocking any subsequent `confirm()` or Roll tap until the engine completes its turn and `readMatchState` transitions to `HUMAN_MOVING` or `WAITING_FOR_ROLL`.

Used Dice Graying Player dice that have been consumed by a sub-move are rendered at `Color.White.copy(alpha = 0.3f)`. Both dice are always shown (using `originalDice` for display); `usedCount = diceToShow.size - remainingDice.size` drives the graying index.

Bearoff Tray Checkers Borne-off checkers are computed as `15 - sum(board[25..49])` (human) and `15 - sum(board[0..24])` (engine), then rendered as stacked checker slots in the respective tray areas.

Winner Detection Previously inverted. `getMatchWinner()` reads `ms.anScore[0]` vs `ms.anScore[1]`, updated by `ApplyGameOver()` in `play.c`. Returns 0 for human win, 1 for engine win.

Bar Re-entry Tap on the bar area (bottom half of the centre bar) sets `src=24` for `ApplySubMove`. Previously the bar had no tap handler and checkers on the bar could not be re-entered.

No Legal Moves When `CommandRoll` finds no legal moves, it calls `TurnDone()` internally. `PlayMove` with an empty move sets `ms.fTurn = !fPlayer`. The fix: `while(fNextTurn) NextTurn(TRUE)` in the `rollDice JNI` function, mirroring the non-GTK loop in `play.c` line 3596.

1.6.3 6.3 Architecture Notes

The single most important lesson from this phase: **every piece of display state must be read from gnubg's own data structures, not reconstructed in Kotlin**. The engine dice problem was a direct consequence of reading `ms.anDice` (transient) instead of the move record (persistent). The move record is the authoritative source for everything that has happened in the game.

1.6.4 6.4 Immediate Next Steps (v0.8.7)

1. Max 5 checkers visual — stack overflow display with count in top checker
2. Doubling cube UI — offer, accept, pass (`CommandDouble`, `CommandTake`, `CommandDrop`)
3. Player panel — score, match length, player names
4. Gammon/backgammon detection on game over
5. Rebuild `.so` with `-Wl, -z, max-page-size=16384` for Android 16 16KB page alignment